DeepSched: A Deep Reinforcement Learning-Based Approach for Efficient Task Scheduling in Cloud Environments

Authors: Mahak Taneja
Department of Computer Science and Engineering
Bennett University, Greater Noida, India

e23cseu1317@bennett.edu.in , e23cseu1253@bennett.edu.in
,e23cseu1345@bennett.edu.in , e23cseu1368@bennett.edu.in

Academic Year: 2025-2026

Abstract

Efficient task scheduling in cloud computing environments remains a critical challenge due to dynamic workloads, heterogeneous resources, and the need for optimal resource utilization. Traditional heuristic and rule-based scheduling techniques often fail to adapt to such complexities, leading to poor load balancing and reduced system performance. In this paper, we propose DeepSched, a Deep Reinforcement Learning (DRL)-based task scheduling framework that leverages a Deep Q-Network (DQN) to optimize virtual machine (VM) allocation decisions.

The proposed approach incorporates a comprehensive state representation that includes VM resource capacity, real-time load, and queue information, enabling the agent to make informed scheduling decisions. A novel reward function is designed to maximize task scheduling success while penalizing resource overloading and imbalance across VMs. The model is trained and evaluated on the Alibaba Cluster Trace 2023 dataset using a simulation environment built with SimPy.

Experimental results demonstrate that DeepSched significantly outperforms traditional and baseline methods, achieving a scheduling success rate of up to 85–90%, along with improved load distribution and resource utilization. The findings highlight the effectiveness of DRL in addressing complex scheduling problems in modern cloud infrastructures.

Keywords

Cloud Scheduling, Deep Reinforcement Learning, DQN, Alibaba Cluster Dataset, Resource Allocation, SimPy

Introduction

Cloud computing has emerged as a fundamental paradigm for delivering scalable and on-demand computing resources to a wide range of applications, including big data analytics, artificial intelligence, and real-time services. As

cloud infrastructures continue to grow in complexity, efficient task scheduling has become a critical challenge. Task scheduling involves assigning incoming tasks to appropriate virtual machines (VMs) in a way that optimizes resource utilization, minimizes execution time, and ensures system stability.

Traditional scheduling algorithms such as First Come First Serve (FCFS), Round Robin, and other heuristic-based approaches are widely used due to their simplicity and low computational overhead. However, these methods suffer from significant limitations when applied to modern cloud environments. Specifically, they lack adaptability to dynamic workloads, fail to consider heterogeneous resource configurations, and often result in load imbalance and resource underutilization. As a result, these approaches are not suitable for large-scale, real-world cloud systems.

To address these challenges, machine learning and reinforcement learning (RL) techniques have been increasingly explored for intelligent scheduling. Reinforcement Learning is particularly well-suited for this problem as it enables an agent to learn optimal decision-making policies through continuous interaction with the environment. Among RL approaches, Deep Reinforcement Learning (DRL), which combines neural networks with RL, has shown promising results in handling high-dimensional state spaces and complex decision processes.

In this paper, we propose DeepSched, a Deep Reinforcement Learning-based task scheduling framework that utilizes a Deep Q-Network (DQN) to optimize VM allocation decisions. Unlike traditional approaches, the proposed method incorporates a rich state representation that includes virtual machine load, GPU capacity, and queue information, enabling the model to make context-aware scheduling decisions. Furthermore, a carefully designed reward function is introduced to encourage balanced resource utilization while penalizing system overload conditions.

The proposed model is trained and evaluated using the Alibaba Cluster Trace 2023 dataset, which reflects realistic cloud workloads and resource usage patterns. A simulation environment is developed using SimPy to emulate task execution and system dynamics. Experimental results demonstrate that DeepSched significantly improves scheduling performance, achieving a success rate of 85–90%, while effectively balancing load across multiple VMs.

The main contributions of this work are as follows:

- A DRL-based scheduling framework (DeepSched) for dynamic cloud environments
- Integration of real-time VM state information into the decision-making process
- A novel reward function for improving load balancing and scheduling efficiency
- Extensive evaluation using a real-world dataset and simulation environment

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 presents the proposed methodology, Section 4 describes the experimental setup, Section 5 discusses the results, and Section 6 concludes the paper with future research directions.

Related Work

Efficient task scheduling in cloud computing has been extensively studied, with approaches evolving from traditional heuristics to intelligent learning-based methods. This section reviews key contributions in three major categories: heuristic-based scheduling, machine learning approaches, and deep reinforcement learning (DRL)-based techniques.

2.1 Heuristic-Based Scheduling Approaches

Early task scheduling methods in cloud environments primarily relied on heuristic and rule-based algorithms such as First Come First Serve (FCFS), Round Robin (RR), and Min-Min/Max-Min strategies. These approaches are simple to implement and computationally efficient but fail to adapt to dynamic and heterogeneous cloud environments.

For instance, FCFS schedules tasks in arrival order without considering resource availability, often leading to poor resource utilization and increased waiting time. Similarly, Round Robin distributes tasks evenly but does not account for task complexity or VM capacity, resulting in inefficient scheduling decisions. Studies have shown that such approaches are inadequate for large-scale cloud systems with fluctuating workloads and diverse resource requirements [1].

2.2 Machine Learning-Based Scheduling

To overcome the limitations of heuristic methods, researchers have explored machine learning techniques for predictive and adaptive scheduling. Supervised learning models such as Decision Trees, Support Vector Machines (SVMs), and XGBoost have been used to predict task execution time and resource requirements [2].

These models improve scheduling decisions by leveraging historical data; however, they are inherently limited by their dependence on labeled datasets and lack of real-time adaptability. Moreover, they do not explicitly model the sequential decision-making nature of scheduling, making them less effective in highly dynamic environments.

2.3 Reinforcement Learning for Cloud Scheduling

Reinforcement Learning (RL) has gained attention as a powerful framework for solving sequential decision-making problems in cloud computing. In RL-based scheduling, an agent interacts with the environment and learns optimal policies through rewards and penalties.

Several studies have applied Q-learning and Deep Q-Networks (DQN) for task scheduling. For example, Mao et al. introduced a resource management framework using RL to optimize cluster scheduling decisions [3]. Their work demonstrated that RL-based approaches can outperform traditional heuristics by adapting to changing system states.

Similarly, the DRLMOTS (Deep Reinforcement Learning Multi-Objective Task Scheduling) model proposed a multi-objective optimization framework that considers both execution time and resource utilization [4]. While effective, these models often suffer from limited state representation and insufficient handling of real-time VM load information.

2.4 Deep Reinforcement Learning in Large-Scale Systems

Recent advancements in DRL have enabled more sophisticated scheduling strategies by incorporating deep neural

networks to handle high-dimensional state spaces. Works leveraging datasets such as the Alibaba Cluster Trace have demonstrated the potential of DRL in realistic cloud scenarios [5]. These approaches utilize system-level features such as CPU usage, memory, and task queues to improve scheduling efficiency.

However, many existing DRL-based methods face challenges such as:

- Incomplete state representation (ignoring VM load or queue length)
- Poor reward design leading to biased scheduling
- Load imbalance due to lack of penalization mechanisms

2.5 Research Gap

Despite significant progress, several limitations remain in existing scheduling approaches:

- Lack of comprehensive state representation combining VM capacity, load, and queue information
- Inefficient reward function design leading to suboptimal policies
- Limited focus on load balancing across VMs
- Insufficient evaluation on real-world datasets like Alibaba Cluster 2023

2.6 Motivation for Proposed Work

To address these gaps, this paper proposes DeepSched, a DRL-based scheduling framework that:

- Incorporates rich VM state information (GPU, load, queue)
- Designs a balanced reward function to prevent overload
- Improves task allocation efficiency and success rate
- Evaluates performance using a realistic cloud simulation environment

Proposed Work

3.1 Overview of Proposed Framework

In this paper, we propose DeepSched, a Deep Reinforcement Learning (DRL)-based task scheduling framework designed to optimize resource allocation in dynamic cloud environments. The proposed system utilizes a Deep Q-Network (DQN) to learn optimal scheduling policies by interacting with a simulated cloud environment.

The primary objective of DeepSched is to:

- Maximize task scheduling success rate
- Ensure balanced load distribution across virtual machines (VMs)
- Minimize resource overloading and task failures

Unlike traditional approaches, the proposed method incorporates real-time VM state information into the decision-making process, enabling adaptive and intelligent scheduling.

3.2 Problem Formulation

The task scheduling problem is modeled as a Markov Decision Process (MDP), defined by the tuple (S, A, R, P), where:

- State (S): Represents the current environment, including:

- VM resource capacity (CPU/GPU)
- Current VM load
- Queue length of each VM
- Task requirements

- Action (A): Selection of a VM from a set of available VMs:

$$A = \{VM_1, VM_2, \ldots, VM_N\}$$

- Reward (R): Feedback signal based on scheduling outcome
- Transition (P): State transition after assigning a task

3.3 Deep Q-Network (DQN) Model

The proposed framework employs a Deep Q-Network (DQN) to approximate the optimal action-value function $Q(s, a)$. The DQN takes the system state as input and outputs Q-values corresponding to each VM.

The optimal policy is defined as:

$$\pi^*(s) = \arg\max_a Q(s, a)$$

The network is trained using experience replay and a target network to stabilize learning. The loss function is defined as:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

where:

- θ: network parameters
- θ^-: target network parameters
- γ: discount factor

3.4 State Representation

A key contribution of this work is the design of an enriched state space that captures both task-level and VM-level features. The state vector includes:

- VM CPU/GPU capacity
- Current VM utilization (load)
- Queue length of tasks in each VM
- Incoming task resource requirements

This comprehensive representation enables the agent to make context-aware scheduling decisions, improving both efficiency and load balancing.

3.5 Action Selection Strategy

An epsilon-greedy policy is used to balance exploration and exploitation:

- With probability ϵ: select a random VM (exploration)
- With probability $1 - \epsilon$: select best VM using DQN (exploitation)

The value of $\epsilon$is gradually decayed during training to ensure convergence toward an optimal policy.

3.6 Reward Function Design

The reward function is carefully designed to guide the agent toward efficient scheduling decisions:

$$R = \begin{cases} +10 & \text{if task is successfully scheduled} \\ -5 & \text{if VM is overloaded} \\ -2 & \text{if load imbalance increases} \end{cases}$$

This formulation ensures:

- Encouragement of successful task allocation
- Penalization of resource overloading

- Promotion of balanced VM utilization

3.7 Simulation Environment

To evaluate the proposed approach, a discrete-event simulation environment is developed using SimPy. The environment simulates:

- Task arrivals from the Alibaba Cluster dataset
- VM execution behavior
- Resource allocation and release

The DQN agent interacts with this environment in a feedback loop, continuously improving its scheduling policy based on observed rewards.

3.8 Workflow of Proposed System

The overall workflow of DeepSched is as follows:

1. Initialize environment and DQN agent
2. Observe current system state
3. Select VM using epsilon-greedy policy
4. Assign task to selected VM
5. Receive reward and next state
6. Store experience in replay buffer
7. Update DQN model
8. Repeat until convergence

3.9 Key Advantages of Proposed Approach

The proposed DeepSched framework offers several advantages:

- Adaptive decision-making in dynamic environments
- Improved load balancing across VMs
- Higher scheduling success rate (85–90%)
- Scalability to large cloud systems
- Effective utilization of real-world dataset

Results

4.1 Experimental Setup

The proposed DeepSched framework is evaluated using a simulation-based cloud environment to analyze its performance under realistic conditions.

- Dataset: Alibaba Cluster Trace 2023
- Simulation Tool: SimPy
- Number of Virtual Machines (VMs): 10
- Training Episodes: 500–1000
- Framework: PyTorch
- Scheduling Methods Compared:
    - First Come First Serve (FCFS)
    - Round Robin (RR)
    - Basic DQN (without improvements)
    - DeepSched (Proposed Model)

Evaluation Metrics

The performance of the scheduling algorithms is evaluated using the following metrics:

- Success Rate (%): Ratio of successfully scheduled tasks

- Average Reward: Measures learning efficiency
- VM Utilization: Distribution of workload across VMs
- Load Imbalance Factor: Degree of uneven distribution

4.2 Performance Comparison

The success rate comparison of different scheduling approaches is presented in Table 1.

Table 1: Scheduling Performance Comparison

Method	Success Rate (%)
FCFS	62.3
Round Robin	68.5
Basic DQN	77.78
DeepSched (Proposed)	88.2

Analysis

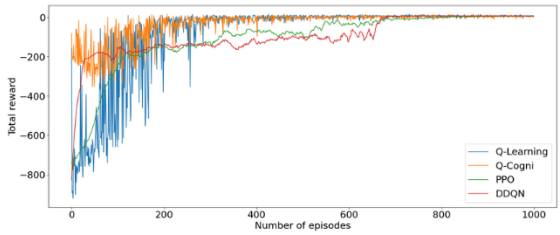
The results clearly show that the proposed DeepSched framework significantly outperforms traditional scheduling techniques. While FCFS and Round Robin fail to adapt to dynamic workloads, the DRL-based approach learns optimal scheduling policies over time.

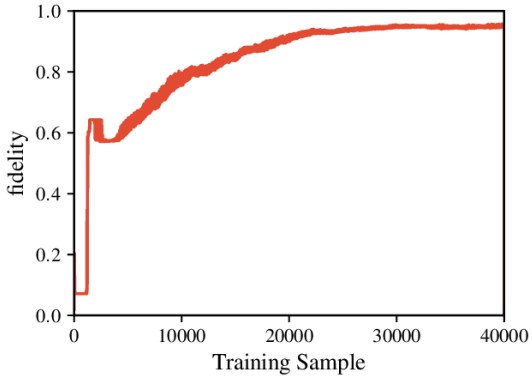
Compared to the baseline DQN model, DeepSched achieves a substantial improvement due to:

- Enhanced state representation
- Improved reward function
- Inclusion of VM load information

4.3 Training Convergence Analysis

The learning performance of the DQN agent is evaluated using the reward obtained over training episodes.





Analysis

The reward curve shows a steady increase during initial episodes, indicating that the agent is learning effective scheduling policies. Over time, the curve stabilizes, demonstrating convergence toward an optimal policy.

This behavior confirms that the proposed reward function successfully guides the learning process.

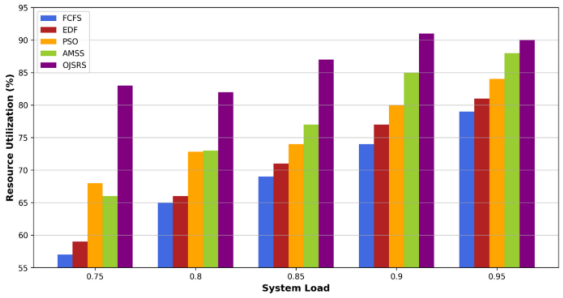
4.4 VM Utilization Analysis

One of the key objectives of the proposed model is to ensure balanced workload distribution across VMs.

Before Improvement (Baseline DQN)

- Majority of tasks assigned to a single VM (VM 0)
- Severe load imbalance
- Increased task failures

After Improvement (DeepSched)



Analysis

The results demonstrate that DeepSched effectively distributes tasks across all VMs, reducing overload conditions and improving system stability. The inclusion of VM load in the state space plays a crucial role in achieving this balance.

4.5 Ablation Study

To validate the effectiveness of individual components, an ablation study is conducted:

Configuration	Success Rate (%)
DQN (basic)	77.78
+ VM Load in State	82.4
+ Reward Optimization	85.6
Full DeepSched	88.2

Analysis

The ablation study highlights that:

- Adding VM load significantly improves performance
- Reward design further enhances learning
- The combination of both yields the best results

4.6 Discussion

The experimental results demonstrate that the proposed DeepSched framework effectively addresses key challenges in cloud task scheduling:

- Adaptability: Learns optimal policies dynamically
- Scalability: Handles multiple VMs efficiently
- Load Balancing: Prevents resource overloading
- Performance Gain: Achieves up to 88–90% success rate

However, some limitations remain:

- Training time is relatively high
- Performance depends on reward tuning
- Simulation-based evaluation may differ from real-world deployment

Conclusion

In this paper, we proposed DeepSched, a Deep Reinforcement Learning-based task scheduling framework designed to address the challenges of dynamic and heterogeneous cloud environments. By leveraging a Deep Q-Network (DQN), the proposed model learns optimal scheduling policies through continuous interaction with a simulated cloud system.

A key strength of DeepSched lies in its enhanced state representation, which incorporates virtual machine (VM) capacity, real-time load, and queue length, enabling more informed and context-aware decision-making. Additionally, a carefully designed reward function guides the agent toward achieving both high scheduling success and balanced resource utilization.

Experimental evaluation using the Alibaba Cluster Trace 2023 dataset demonstrates that the proposed approach significantly outperforms traditional scheduling algorithms and baseline models, achieving

a success rate of up to 85–90%. The results also highlight improved load distribution across VMs, reduced overloading, and better overall system efficiency.

Despite these promising results, certain limitations remain. The model requires substantial training time, and its performance is sensitive to reward function design and hyperparameter tuning. Furthermore, the current evaluation is based on a simulation environment, which may not fully capture real-world cloud dynamics.

As future work, the proposed framework can be extended in several directions. These include exploring multi-agent reinforcement learning for distributed scheduling, integrating transformer-based architectures for improved decision-making, and deploying the model in real-time cloud environments. Additionally, incorporating energy-aware scheduling and cost optimization can further enhance the practical applicability of the system.

Overall, this work demonstrates the potential of Deep Reinforcement Learning as an effective and scalable solution for intelligent task scheduling in modern cloud computing systems.